

电控第一次作业说明文档

姓名：支亦韬

学号：3250103624

芯片型号：STM32F103C8T6

开发环境：STM32CubeMX + CMake + VS Code + Ozone + DapLink

一、工程概述

本次作业在一个 CubeMX 工程中完成三题。工程配置如下：

1. Toolchain/IDE 选择 CMake，勾选 Generate Under Root，生成代码。
 2. 用发放的 CMakeLists.txt 替换 CubeMX 生成的模板（保持 user_folders "Tasks"），并在根目录建立 Tasks/inc 和 Tasks/src 后再执行 CMake Configure。
 3. main.c 仅在 USER CODE 区域包含头文件并调用初始化，while(1) 保持为空。
 4. 业务代码全部写在 Tasks 文件夹中。IWDG 从第一题起保持启用。
-

二、任务一：GPIO 点灯

2.1 配置与代码

在 CubeMX 中，将 F103 最小系统板的 PC13 引脚配置为推挽输出（Output Push Pull），初始电平设为低电平。在 Tasks/src/tasks.c 的 Tasks_Init() 函数中实现：

```
void Tasks_Init(void)
{
    HAL_GPIO_WritePin(GPIOC, GPIO_PIN_13, GPIO_PIN_RESET);
}
```

main.c 中在 USER CODE BEGIN 2 调用了 Tasks_Init()。按照任务一要求，此处未调用 HAL_IWDG_Refresh。

2.2 观察现象

程序下载运行后，板载蓝灯亮起。由于未喂狗，IWDG 约 2 秒后超时触发系统复位，蓝灯闪烁/熄灭后重新亮起，不断循环。现象符合第一题预期，灯亮照片见附录图1。

三、任务二：定时器与喂狗

3.1 定时器配置与计算过程

在 CubeMX 中启用 TIM2，时钟源选择 Internal Clock，并开启 TIM2 全局中断。

PSC 和 ARR 计算过程：

- 由时钟树可知，APB1 分频系数为 2（不为1），因此 APB1 Timer clocks 是 APB1 外设时钟的 2 倍，即 72MHz。
- 目标中断周期为 1ms，公式： $(PSC+1)*(ARR+1)/\text{定时器时钟} = 0.001s$
- 代入计算： $(PSC+1)*(ARR+1) = 72,000,000 * 0.001 = 72000$
- 取 $PSC = 71$ （分频后时钟 1MHz）， $ARR = 999$ （计数 1000 次）。
- 验证： $(71+1)*(999+1)/72,000,000 = 72*1000/72000000 = 0.001s = 1ms$ 。

3.2 回调代码实现

全局变量必须定义为 `volatile uint32_t tick`。在 `Tasks/src/tasks.c` 中重写定时器更新回调（全工程仅此一份，删除了 CubeMX 可能生成的同名函数）：

```
volatile uint32_t tick = 0;

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        tick++;
        HAL_IWDG_Refresh(&hiwdg);
    }
}
```

在 `Tasks_Init()` 中调用 `HAL_TIM_Base_Start_IT(&htim2)`；启动定时器中断。定时器溢出触发中断后，HAL 库的中断服务程序会自动调用 `HAL_TIM_PeriodElapsedCallback` 这个弱函数，我们在 Tasks 中重写了它，因此中断发生时实际执行的就是我们写的 `tick++` 和喂狗逻辑。

3.3 观察现象

Ozone 全速运行程序，在 Watch 窗口监视 `tick` 变量，可见 `tick` 每秒稳定增加约 1000，芯片未发生复位。截图见附录图2。

四、任务三：看门狗复位现象

4.1 参数计算与代码修改

IWDG 保持启用，超时设置约 2 秒。F103 的 LSI 频率按手册取约 40kHz。

- 超时时间公式： $\text{超时} = (\text{Reload} + 1) * \text{分频} / \text{LSI}$
- 取分频 = 64, Reload = 1249, 则： $(1249 + 1) * 64 / 40000 = 2.0s$ 。

在 `Tasks/src/tasks.c` 中，将回调函数内的 `HAL_IWDG_Refresh(&hiwdg)`；注释掉，重新编译下载。

4.2 观察现象

Ozone 全速运行，Watch 窗口监视 `tick`：

- `tick` 仍每 1ms 自增 1。
- 约 2 秒后，`tick` 涨到 2100 左右（由于芯片 LSI 实际频率存在个体误差，略高于理论值 2000 属于正常现象）。
- 随后 `tick` 瞬间归零（复位后启动代码清空全局变量），并重新开始上涨，不断循环。

截图见附录图3。

五、遇到的问题与解决

在此次作业中，除了代码逻辑，我在环境和工具配置上耗费了大量精力，主要问题及解决如下：

1. DapLink 驱动无法识别：

- **现象：** Ozone v3.26原版仅支持J-Link,识别不到我接入的daplink调试板，导致无法烧录
- **解决：** 上次环境配置漏掉了，替换破解版的JLink_x64.dll后就可以识别到，然后再借助J-Link注册机得到一个license，将其写入JLinkLicenseManager即可正常烧录。

2. Tasks 目录进编译的前提：

- **现象：**代码写好了但提示函数未定义。
- **解决：**必须在 CMake:Configure **之前**建好 `Tasks/inc` 和 `Tasks/src` 目录，否则 CMake 扫描不到，Tasks 不会进入编译。

3. STMCubeMX里设置错误导致SWD被锁定：

- **现象：**烧录一次进去后再烧录就会显示Connection failed(还是error? 忘了)
- **解决：**经Q群里消息的提醒，我用pyocd来实现全片擦除，清除了原来的程序（比在 cubeprogrammer里识别不到daplink更便利）。另外就是重新来一遍，注意将system core里sys的debug选上serial wire，这样就能重复烧录了。

4. Ozone Timeline参数设置：

- **现象：**一开始忘设置参数，运行后没有图像；设置tick参数后图像只有一条直线在不断平移上升。
- **解决：**取消勾选Autoscroll，让取景框固定，数据会不断在原来的基础上延伸，形成一条上升的斜线。另外要设置Time scale、y轴取值范围和采样频率以使图像完整显示。

附录

图1：任务一 GPIO 点灯照片

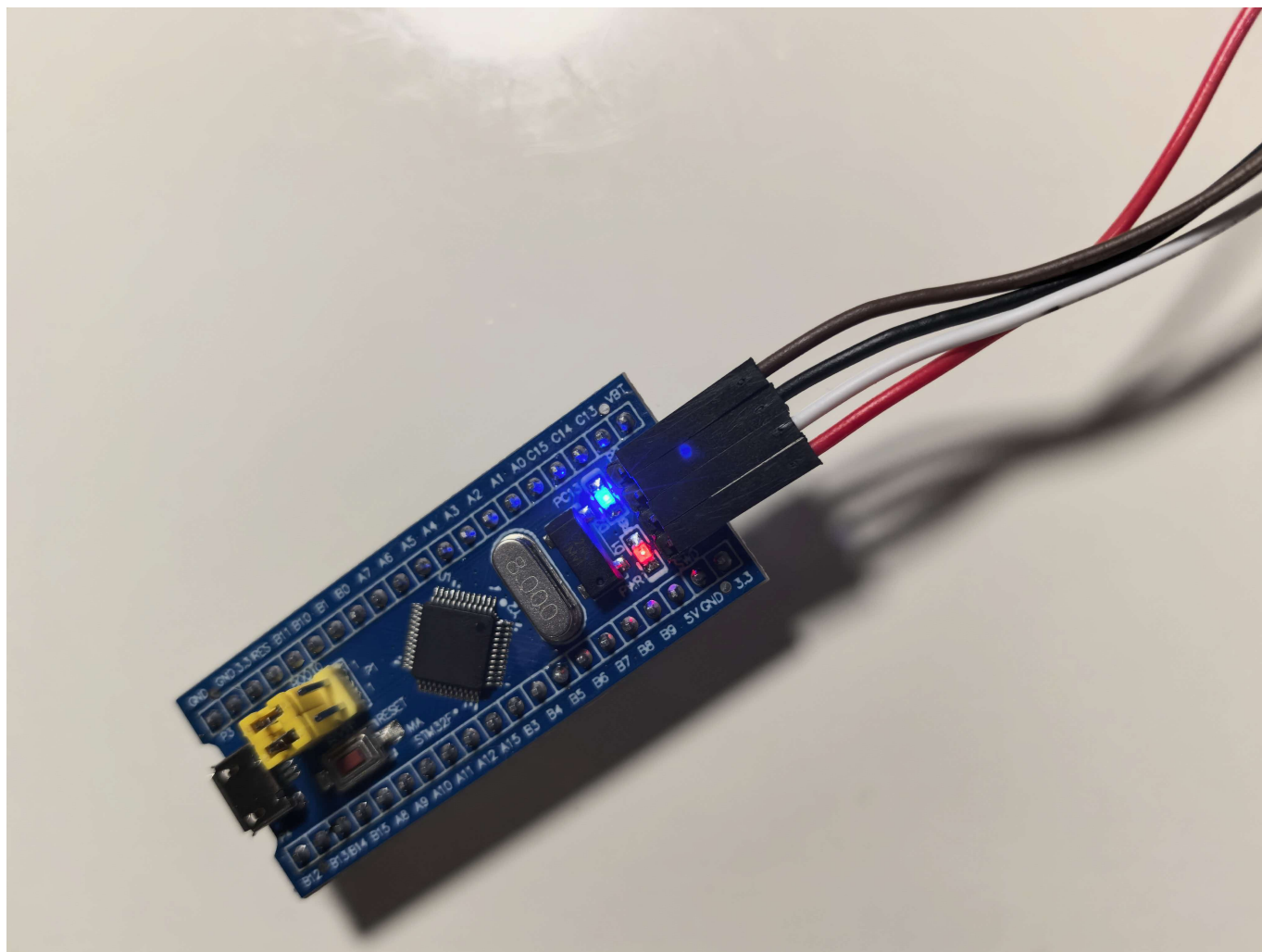


图2：任务二 定时器 tick 持续增加截图

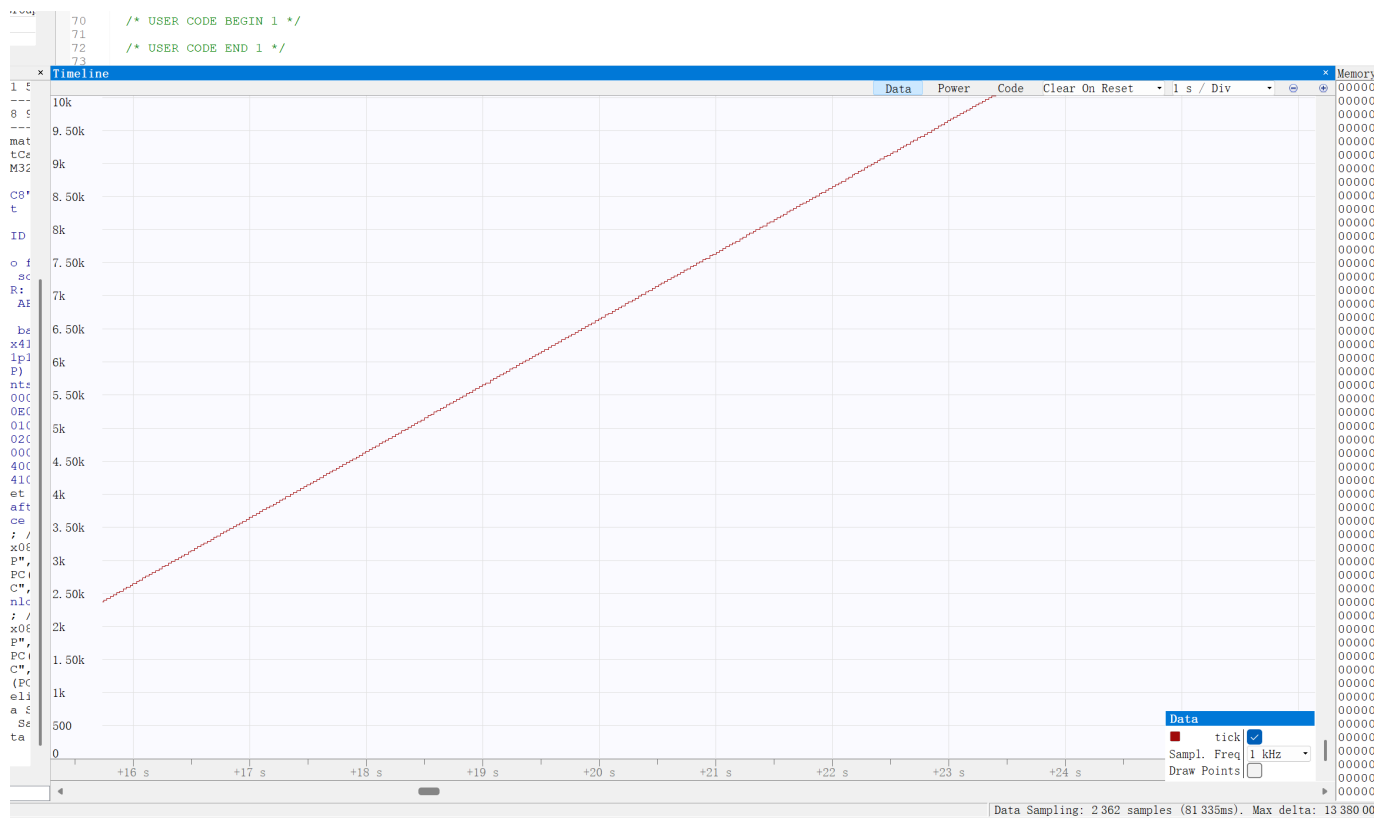


图3：任务三 看门狗复位 tick 归零截图

